

KuCampusBite

Coding Standards

To maintain consistency, readability, and maintainability in the KuCampusBite, the following coding standards have been defined. These standards apply across all modules (front-end, back-end, and database).

1. Naming Conventions

Variables :

- Use camelCase for variables.
- Names must be meaningful and descriptive.
- Example:

```
1 let customerName;  
2 let orderTotal;  
3 let isPaymentComplete;  
4
```

Functions & Methods :

- Use camelCase.
- Start with verbs to indicate the action performed.
- Example:

```
1 function placeOrder() {}  
2 function calculateBill() {}  
3 function fetchMenuItems() {}
```

Constants

- Use **UPPER_CASE** with underscores(optional) or camelCase.
- Example:

```
1 const MAX_CART_ITEMS = 20;  
2 const DISCOUNT_RATE = 0.1;
```

Database Tables &

Columns :

- Tables: plural names → users, orders, menu_items.

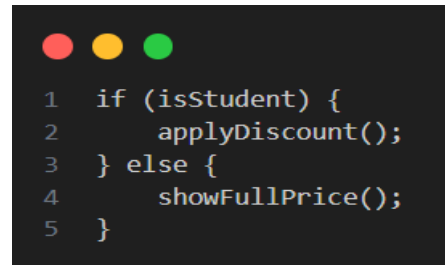
- Columns: snake_case → order_id, customer_id, created_at.

File Names :

- Frontend files: lowercase with hyphens → order-summary.js, user-login.css.
- Backend files: lowercase with underscores → order_controller.js, db_connection.js.

2. Code Formatting

- **Indentation:** 4 spaces (no tabs).
- **Line length:** ≤ 100 characters.
- **Braces:** Opening brace { on the same line.
- **One statement per line.**
- Add **blank lines** between functions and logical code blocks.



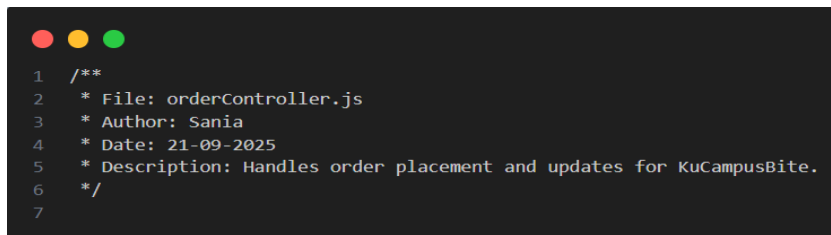
```
1  if (isStudent) {
2      applyDiscount();
3  } else {
4      showFullPrice();
5  }
```

3. Commenting Standards

File Header Comment

Each file must include:

- File name
- Author
- Date
- Description



```
1  /**
2   * File: orderController.js
3   * Author: Sania
4   * Date: 21-09-2025
5   * Description: Handles order placement and updates for KuCampusBite.
6   */
7
```

Single-Line Comment

- Single-line comments must be written in clear and simple English.
- Place comments above the code and keep them short and meaningful.

4. Error Handling

Frontend

- Validate user inputs (e.g., empty fields, invalid formats).
- Show user-friendly messages, not technical errors.

- Example: “Invalid email address. Please try again.”

Backend

- Use try-catch for all database and API calls.
- Return consistent JSON responses:

```
1  {
2    "success": false,
3    "message": "Unable to process request. Please try again."
4  }
5
```

Database

- Use constraints (NOT NULL, UNIQUE, FOREIGN KEY).
- Prevent duplicate entries (e.g., same email registration).

5. File & Project Structure

Frontend:

```
1  /frontend
2  /Authentication
3    login.html
4    login.js
5    signup.html
6    signup.js
7    style.css
8
9  /HomePage
10   KUCampusBite.html
11   KUCampusBite.css
12   canteen.js
13   home.js
14   search.js
15
16 /CanteenDetail
17   CanteenDetail.html
18   CanteenDetail.css
19   CanteenDetail.js
20
21 /ItemDetail
22   ItemDetail.html
23   ItemDetail.css
24   ItemDetail.js
25   review.js
26
27 /AllMenuPage
28   allmenu.html
29   style.css
30   allmenu.js
31
```

```

32: /AllCanteenPage
33:   allcanteen.html
34:   style.css
35:   allcanteen.js
36:
37: /Cart
38:   cart.html
39:   cart.css
40:   cart.js
41:   addToCart.js
42:
43:
44: /Checkout
45:   checkout.html
46:   checkout.css
47:   checkout.js

```

Backend (Node.js):

```

1  /backend
2    /controllers
3      orderController.js
4      userController.js
5    /models
6      Order.js
7      User.js
8      MenuItem.js
9    /routes
10     orderRoutes.js
11     userRoutes.js
12   /config
13     db.js
14     server.js
15   /middlewares
16     authMiddleware.js

```

Database:

Collections:

- **Users:**
 - ID, Name, Email, Password, Role.
- **menu_items:**
 - ID, Name, Category, Price, Availability.

- **Orders:**

ID, User_ID, Items (Menu_Item_ID, Name, Quantity, Price], Total_Amount, Status.

- **Feedback:**

ID, User_ID, Message, Rating.

6. Best Practices

- Keep functions short and focused on a single task.
- Use code reusability (avoid duplication).
- Commit code frequently with meaningful messages.
- Follow **Git commit convention**:
 1. **feat**: for new features
 2. **fix**: for bug fixes
 3. **docs**: for documentation updates
 4. **refactor**: for restructuring